EET 303 MODULE-5

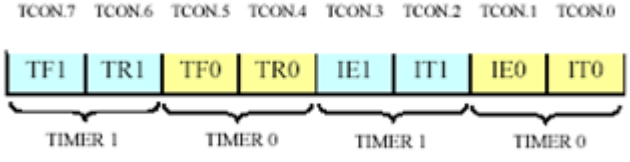
SYLLABUS

8051 Timer/counter programming - Serial port programming - Interrupt programming in assembly language and embedded C.
Interfacing –ADC - DAC and temperature sensor

- 8051 has two 16-bit programmable UP timers/counters.
- They can be configured to operate either as timers or as event counters. The names of the two counters are T0 and T1 respectively.
- TH0/TL0 is the 16-bit register of Timer 0 and TH1/TL1 is the 16-bit register of Timer 1.
- Timer Special Function Registers are
    - TCON TimerControl
    - TMOD TimerMode

5.1.1 Timer SFRs:

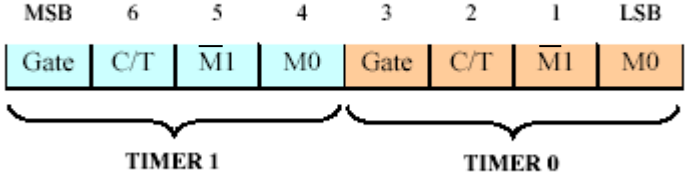
TCON



TCONSFRandits individualbits

- IT0/IT1:UsedfortimerInterrupts
- IE0/IE1:UsedforexternalInterrupts
- TR0/TR1:Timer 0/1 run control bit. Set to 1 to start the timer / counter.
- TF0/TF1:Timer0/1 overflow flag. It is set when timer rolls from all 1s to 0s

TMOD



TMODSFRanditsindividualbits

- M0/M1:setstheModeoftherespectivetimer

M1	M0	Mode
0	0	Mode 0
0	1	Mode 1
1	0	Mode 2
1	1	Mode 3

- C/$\overline{T}$:ExternalCounter/InternalTimerselect
  1=Counter,0= Timer
- Gate:Whenset(1),timerrunsonly whenrespectiveINTinputis high.

5.1.2 Timer Operating Modes:

Four operating modes are there:

- Mode 0:13 bittimer
- Mode 1:16-bittimer
- Mode 2:8-Bitautoreload
- Mode 3:Splittimermode

Timer Mode 0: (13 bit Mode)

- In this mode, the timer is used as a 13-bit UP counter as follows.

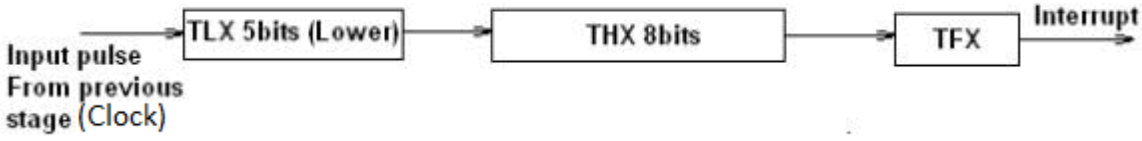


Fig. 4.6 Operation of Timer on Mode-0

- The lower 5 bits of TLX and 8 bits of THX are used for the 13 bit count. Upper 3 bits of TLX are ignored.
- When the counter rolls over from all 0's to all 1's, TFX flag is set and an interrupt is generated.

Timer Mode1:(16-bit mode)

- All16 bitsof thetimer(TH0/TL0,TH1,TL1)areused.
- Maximumcountis65,536
- This mode is similar to mode-0 except for the fact that the Timer operates in 16-bit mode.

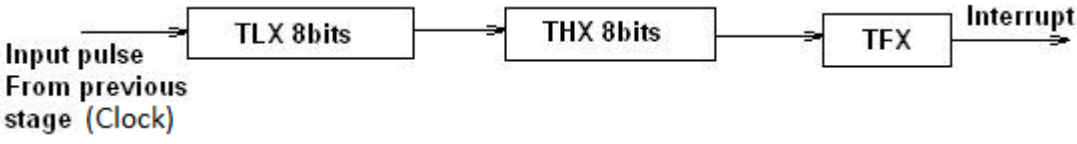
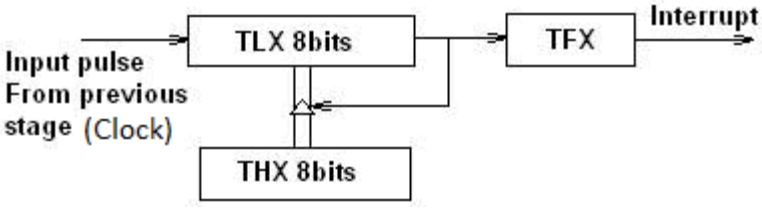


Fig Operation of Timer in Mode 1

Timer Mode-2: (Auto-Reload Mode)

- This is a 8 bit counter/timer operation.
- Counting is performed in TLX while THX stores a constant value.
- In this mode when the timer overflows i.e. TLX becomes FFH, it is fed with the value stored in THX.
- For example if we load THX with 50H then the timer in mode 2 will count from 50H to FFH. After that 50H is again reloaded.



Timer Mode-3 (Split mode)

- Timer0 in mode-3 establishes TL0 and TH0 as two separate counters.
- Timer 1 in mode-3 simply holds its count.
- Control bits TR1 and TF1 are used by Timer-0 (higher 8 bits) (TH0).
- TR0 and TF0 are available to Timer-0 lower 8 bits(TL0).

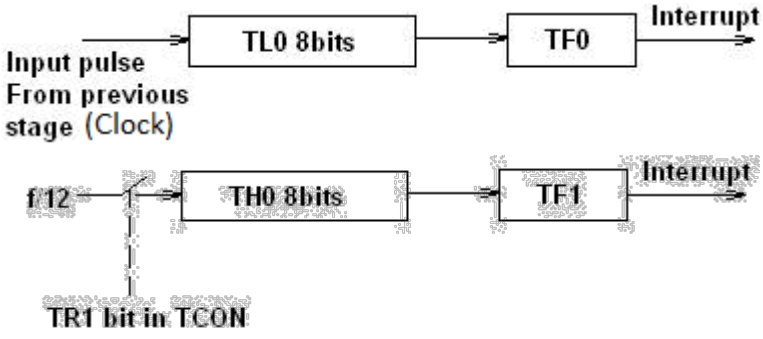


Fig Operation of Timer in Mode 3

5.3 Timer / Counter Programming :

In order to program 8051 timers, it is important to know the calculation of initial count value to be stored in the timer register. The calculations are as follows.

In any mode, Timer Clock Frequency. = Master Clock Frequency/12
Timer Clock period = 1/Timer Clock Frequency.

a) Mode 1 (16 bit timer/counter)

Value to be loaded in decimal, n = 65536 – (Delay Required/Timer clock period)
Convert the answer into hexadecimal and load onto THx and TLx register.
(65536D = FFFFH+1)

b) Mode 0 (13 bit timer/counter)

Value to be loaded in decimal = 8192 – (Delay Required/Timer clock period)
Convert the answer into hexadecimal and load onto THx and TLx register.
(8192D = 1FFFH+1)
c) Mode 2 (8 bit auto reload)

Value to be loaded in decimal = 256 – (Delay Required/Timer clock period)
Convert the answer into hexadecimal and load onto THx register. Upon starting the timer this value from THx will be reloaded to TLx register.
(256D = FFH+1)

Steps for programming timers in 8051

Mode 1:

- Load the TMOD value register indicating which timer (0 or 1) is to be used and which timer mode is selected.
- Load registers TL and TH with initial count values.
- Start the timer by the instruction "SETB TR0" for timer 0 and "SETB TR1" for timer 1.
- Keep monitoring the timer flag (TF) with the "JNB TFx,target" instruction to see if it is raised. Get out of the loop when TF becomes high.
- Stop the timer with the instructions "CLR TR0" or "CLR TR1", for timer 0 and timer 1, respectively.
- Clear the TF flag for the next round with the instruction "CLR TF0" or "CLR TF1", for timer 0 and timer 1, respectively.
- Go back to step 2 to load TH and TL again.

Mode 0:

The programming techniques mentioned here are also applicable to counter/timer mode 0. The only difference is in the number of bits of the initialization value. (13 bits)

Mode 2:

- Load the TMOD value register indicating which timer (0 or 1) is to be used; select timer mode 2.
- Load TH register with the initial count value. As it is an 8-bit timer, the valid range is from 00 to FFH.
- Start the timer.
- Keep monitoring the timer flag (TFx) with the "JNB TFx,target" instruction to see if it is raised. Get out of the loop when TFx goes high.
- Clear the TFx flag.
- Go back to step 4, since mode 2 is auto-reload.

Example#1: Write a program to continuously generate a square wave of 2 kHz frequency on pin P1.5 using timer 1. Assume the crystal oscillator frequency to be 12 MHz.

The period of the required square wave is T = 1/(2 kHz) = 500 μs.
Delay required for each half cycle = 250 μs.
Timer Clock frequency = Master Frequency/12
= 12MHz/12 =1 MHz
Timer Clock period = 1/ Timer Clock frequency = 1/1MHz = 1 μs.

Value to be loaded in decimal, n = 65536 – (250 µs /1 µs)
=65536 – (250) = 65286 = FF06h

TL = 06h and TH = 0FFh.

```
        MOV TMOD,#10        ;Timer 1, mode 1
AGAIN: MOV TL1,#06h         ;TL0 = 06h
        MOV TH1,#FFh        ;TH0 = FFh
        SETB TR1            ;Start timer 1
BACK:  JNB TF1,BACK         ;Stay until timer rolls over
        CLR TR1             ;Stop timer 1
        CPL P1.5            ;Complement P1.5 to get High, Low
        CLR TF1             ;Clear timer flag 1
        SJMP AGAIN          ;Reload timer
```

Example#2. Write a 8051 C program to toggle all the bits of port P1 continuously with 1mS delay in between. Use Timer 0, 16-bit mode to generate the delay.
Solution:

Required delay = 1mS = 1000µS.

Let crystal frequency be 12MHz.
Then,
Timer Clock Frequency. = Master Clock Frequency/12
= 12MHz/12 = 1MHz

Therefore, Timer Clock period = 1/Timer Clock Frequency.
= 1/1MHz = 1µS.

Value to be loaded in decimal, n = 65536 – (Delay Required/Timer clock period)
= 65536- (1000µS/1µS)
=65536 -1000 = 64536 = FC18h
Therefore load 18 to register TL0 and load FC to register TH0 to get the required delay.

TMOD register initialization:
The value to be loaded in TMOD register to selecttimer 0 in mode0 = 0000 0001 =01h

Program:

```
#include <reg51.h>
void T0Delay(void);
void main(void)
  {
   while(1)              //repeat forever
    {
      P1=0x55;           //toggle all bits of P1
      T0Delay();
      P1=0xAA;           //toggle all bits of P1
      T0Delay();
    }
  }

void T0Delay()
  {
    TMOD=0x01;           //Timer 0, Mode 1
    TL0=0x18;            //load TL0
    TH0=0xFC;            //load TH0
    TR0=1;               //turn on T0
    while(TF0==0);       //wait for TF0 to roll over
    TR0=0;               //turn off T0
    TF0=0;               //clear TF0
  }
```


Example#3. Write an 8051 C program to toggle only bit P1.5 continuously every 50 mS. Use Timer 1, mode 1 (16-bit) to create the delay.

Required delay = 50mS = 50000μS.

Let crystal frequency be 12MHz.
Then,
Timer Clock Frequency. = Master Clock Frequency/12
= 12MHz/12 = 1MHz

Therefore, Timer Clock period = 1/Timer Clock Frequency.
= 1/1MHz = 1μS.

Value to be loaded in decimal, n = 65536 – (Delay Required/Timer clock period)
= 65536- (50000μS/1μS)
=65536-50000 = 15536 = 3CB0h
Therefore load B0 to register TL1 and load 3C to register TH1 to get the required delay.

TMOD register initialization:
The value to be loaded in TMOD register to selecttimer 0 in mode0 = 0001 0000 =10h

Program:

```
#include <reg51.h>
void Delay(void);
sbit mybit=P1^5;
void main(void)
  {
   while(1)
    {
      mybit=~mybit;   //toggle P1.5
      Delay();        //Timer 1 , mode 1 (16-bit)
    }
  }
void Delay(void)
  {
    TMOD=0x10 ;       //Timer 1 , mode 1 (16-bit)
    TL1=0xB0;         //load TL1
    TH1=0x3C;         //load TH1
    TR1=1;            //turn on T1
    while(TF1==0);    //wait for TF1 to roll over
    TR1=0;            //turn off T1
    TF1=0;            //clear TF1
  }
```


5.2 SERIAL PORT PROGRAMMING

- The 8051 supports a full duplex serial port.
- Three special function registers support serial communication.

1**. SBUF Register**: Serial Buffer (SBUF) register is an 8-bit register. It has separate SBUF registers for data transmission and for data reception. For a byte of data to be transferred via the TXD line, it must be placed in SBUF register. Similarly, SBUF holds the 8-bit data received by the RXD pin and read to accept the received data.

2. **SCON register**: The contents of the Serial Control (SCON) register are shown below. This register contains mode selection bits, serial port interrupt bit (TI and RI) and also the ninth data bit for transmission and reception (TB8 and RB8).

D7	D6	D5	D4	D3	D2	D1	D0
SM0	**SM1**	**SM2**	**REN**	**TB8**	**RB8**	**TI**	**RI**

- SM0 (SCON.7) : Serial communication mode selection bit
- SM1 (SCON.6) : Serial communication mode selection bit

SM0	SM1	Mode	Description	Baud rate
0	0	Mode 0	8-bit shift register mode	Fosc / 12
0	1	Mode 1	8-bit UART	Variable (set by timer 1)
1	0	Mode 2	9-bit UART	Fosc/ 32 or Fosc/64
1	1	Mode 3	9-bit UART	Variable (set by timer 1)

- SM2 (SCON.5) : Multiprocessor communication bit. In modes 2 and 3, if set this will enable multiprocessor communication.
- REN (SCON.4) : Enable serial reception
- TB8 (SCON.3) : This is 9th bit that is transmitted in mode 2 & 3.
- RB8 (SCON.2) : 9th data bit is received in modes 2 & 3.
- TI (SCON.1) : Transmit interrupt flag, set by hardware must be cleared by software.
- RI (SCON.0) : Receive interrupt flag, set by hardware must be cleared by software.

3. **PCON register:**The SMOD bit (bit 7) of PCON register controls the baud rate in asynchronousmode transmission.

Power mode Control (PCON) Register							
D7	D6	D5	D4	D3	D2	D1	D0
SMOD	**--**	**--**	**--**	**GF1**	**GF0**	**PD**	**IDL**

- SMD (PCON.7): Serial rate modify bit. Set to 1 by program to double baud rate using timer 1 for modes 1, 2, and 3. cleared by program to use timer 1 baud rate.

5.2.1 SERIAL COMMUNICATION MODES

1. Mode 0

In this mode serial port runs in synchronous mode. The data is transmitted and received through RXD pin and TXD is used for clock output. In this mode the baud rate is 1/12 of clock frequency.

2. Mode 1

In this mode SBUF becomes a 10 bit full duplex transceiver. The ten bits are 1 start bit, 8 data bit and 1 stop bit. The interrupt flag TI/RI will be set once transmission or reception is over. In this mode the baud rate is variable and is determined by the timer 1 overflow rate.

Baud rate = [2^{smod}/32] x Timer 1 overflow Rate

= [2^{smod} /32] x [Oscillator Clock Frequency] / [12 x [256 – [TH1]]]

3. Mode 2

This is similar to mode 1 except 11 bits are transmitted or received. The 11 bits are, 1 start bit, 8 data bit, a programmable 9th data bit, 1 stop bit.

Baud rate = $[2^{smod}/64]$ x Oscillator Clock Frequency

4. Mode 3

This is similar to mode 2 except baud rate is calculated as in mode 1

Write a C program for the 8051 to transfer the letter "A" serially at 4800 baud continuously. Use 8-bit data and 1 stop bit.

Solution:

```
#include <reg51.h>
void main(void)
  {
    TMOD=0x20;              //use Timer 1,8-BIT auto-reload
    TH1=0xFA;               //4800 baud rate
    SCON=0x50;
    TR1=1;
    while(1)
      {
        SBUF='A';           //place value in buffer
        while(TI==0);
        TI=0;
      }
  }
```

data, 1 stop bit. Do this continuously.

Solution:

```
#include <reg51.h>
void SerTx(unsigned char);
void main(void)
  {
    TMOD=0x20;          //use Timer 1,8-BIT auto-reload
    TH1=0xFD;           //9600 baud rate
    SCON=0x50;
    TR1=1;              //start timer
    while(1)
      {
        SerTx('Y');
        SerTx('E');
        SerTx('S');
      }
  }
void SerTx(unsigned char x)
  {
    SBUF=x;             //place value in buffer
    while(TI==0);       //wait until transmitted
    TI=0;
  }
```

Program the 8051 in C to receive bytes of data serially and put them in P1. Set the baud rate at 4800, 8-bit data, and 1 stop bit.

Solution:

```
#include <reg51.h>
void main (void)
  {
    unsigned char mybyte;
    TMOD=0x20;                  //use Timer 1,8-BIT auto-reload
    TH1=0xFA;                   //4800 baud rate
    SCON=0x50;
    TR1=1;                      //start timer
    while(1)                    //repeat forever
      {
        while(RI==0);           //wait to receive
        mybyte=SBUF;            //save value
        P1=mybyte;              //write value to port
        RI=0;
      }
  }
```

5.3.1 Types of Interrupts in 8051

- When an interrupt event occurs, the microcontroller pause its current program and attend to the interrupt by executing an **Interrupt Service Routine (ISR).**
- At the end of the ISR, the microcontroller returns to the program it had pause and continue its normal operations.

The 8051 microcontroller has five sources of interrupts:

1. Timer 0 overflow interrupt- TF0
2. Timer 1 overflow interrupt- TF1
3. External hardware interrupt- INT0
4. External hardware interrupt- INT1
5. Serial communication interrupt- RI/TI

External Hardware Interrupts :(INT0 and INT1)

- 8051 microcontrollers consists of two external hardware interrupts: INT0 and INT1.
- These can be edge triggered or level triggered.
- In level triggering, the low at the interrupt pin enables the interrupt.
- In edge triggering, the high to low transition enables the edge triggered interrupt.
- This edge triggering or level triggering is decided by the TCON register.

Internal Interrupts:(RI/TI , TF0 and TF1)

- Internallygeneratedinterruptscanbefromeithertimer,orfromtheserialinterface.
- Theserialinterfacecausesinterruptsduetoareceiveevent(RI)ordue toatransmitevent(TI).
- Thereceiveeventoccurswhentheinputbufferofthe serialline(sbufin)isfullandabyteneedstobereadfromit.
- Thetransmitevent indicatesthatabytehasbeensentanewbytecanbewrittentooutputbufferof theserialline(sbufout).

- 8051timersalwayscountup.Whentheircounterrollsoverfromthemaximum countto0000,theysetthecorrespondingtimerflagTF1orTF0inTCON and aninterruptoccurs.

5.3.2 Interrupt Vectors

- Vector Address is the starting address of Interrupt service routine of a particular interrupt.
- Vector address means the address by which the interrupt service routine is stored.
- Thefollowingtablegives thevectoraddresses of various interrupts.

ExternalInterrupt0	0003H
Timer0Overflow	000BH
ExternalInterrupt1	0013H
Timer1Overflow	001BH
SerialInterface	0023H

5.3.3 SequenceofEventsafteraninterrupt

Whenanenabledinterruptoccurs,

1.ThePCissavedonthestack.

2.Otherinterruptsoflowerpriorityandsamepriorityaredisabled.

3.Exceptfortheserialinterrupt,thecorrespondinginterruptflagiscleared.

4.PCisloadedwiththevectoraddresscorrespondingtotheinterrupt.

5. Then interrupt service routine will be executed

When processorexecutes instruction'RETI"

1.PCisrestoredbypoppingthestack.

2.Interruptstatusisrestoredtoitsoriginalvalue.(Sameandlowerpriority interruptsrestoredtooriginalstatus).

5.3.4 Special Function Registers related to interrupt:

1. **Interrupt Enable (IE) Register:**

- This register is responsible for enabling and disabling the interrupt.
- It is a bit addressable register in which EA must be set to one for enabling interrupts.
- The corresponding bit in this register enables particular interrupt like timer, external and serial inputs.
- In the below IE register, bit corresponding to 1 activates the interrupt and 0 disables the interrupt.

EA	–	–	ES	ET1	EX1	ET0	EX0

EA	IE.7	Disables all interrupts. If EA = 0, no interrupt will be acknowledged. If EA = 1, interrupt source is individually enable or disabled by setting or clearing its enable bit.
-	IE.6	Not implemented, reserved for future use*.
-	IE.5	Not implemented, reserved for future use*.
ES	IE.4	Enable or disable the Serial port interrupt.
ET1	IE.3	Enable or disable the Timer 1 overflow interrupt.
EX1	IE.2	Enable or disable External interrupt 1.
ET0	IE.1	Enable or disable the Timer 0 overflow interrupt.
EX0	IE.0	Enable or disable External Interrupt 0.

- Each and every interrupt has a priority level by which the microcontroller serves the interrupt in a predefined manner and its order is INT0, TF0, INT1, TF1, and SI.
- Each interrupt source can be programmed to have one of the two priority levels by setting (high priority) or clearing (low priority) a bit in the IP (Interrupt Priority) Register.
- This allows the low priority interrupt to interrupt the high-priority interrupt, but prohibits the interruption by another low-priority interrupt.
- Similarly, the high-priority interrupt cannot be interrupted.

IP : Interrupt Priority Register (Bit Addressable)

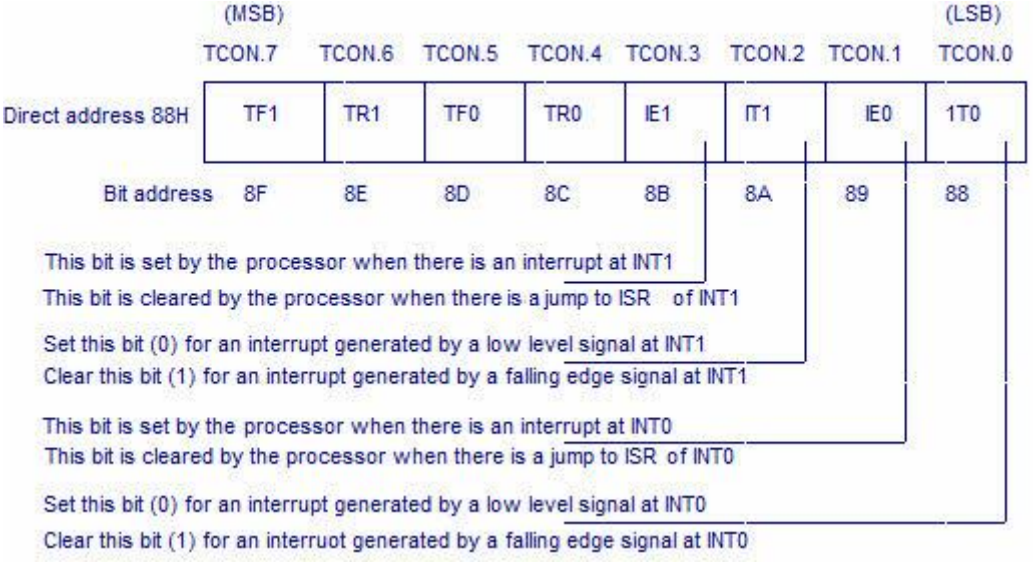
If the bit is 0, the corresponding interrupt has a lower priority and if the bit is the corresponding interrupt has a higher priority.

–	–	–	PS	PT1	PX1	PT0	PX0

-	IP.7	Not implemented, reserved for future use*.
-	IP.6	Not implemented, reserved for future use*.
-	IP.5	Not implemented, reserved for future use*.
PS	IP.4	Defines the Serial Port interrupt priority level.
PT1	IP.3	Defines the Timer 1 Interrupt priority level.
PX1	IP.2	Defines External Interrupt priority level.
PT0	IP.1	Defines the Timer 0 interrupt priority level.
PX0	IP.0	Defines the External Interrupt 0 priority level.

3. TCON Register:

- In addition to the above two registers, the TCON register specifies the type of external interrupt to the 8051 microcontroller, as shown in the figure.
- The two external interrupts, **whether edge or level triggered**, specify by this register by a set, or cleared by appropriate bits in it.



5.3.5 Interrupt Programming in 8051

While programming interrupts, first thing to do is to specify the microcontroller which interrupts must be served. This is done by configuring the Interrupt Enable (IE) register which enables or disables the various available interrupts.

To enable any of the interrupts, first the EA bit must be set to 1. After that the bits corresponding to the desired interrupts are enabled. ET0 and ET1 bits are used to enable the Timer Interrupts 0 and 1 respectively. EX0 and EX1 are used to enable the external interrupts 0 and 1. ES is used for serial interrupt.

EA bit acts as a lock bit. If any of the interrupt bits are enabled but EA bit is not set, the interrupt will not function. By default all the interrupts are in disabled mode.

Note that the IE register is bit addressable and individual interrupt bits can also be accessed.

For example –

IE = 0x81; enables External Interrupt0 (EX0)

IE = 0x88; enables Serial Interrupt

Setting the bits of IE register is necessary and sufficient to enable the interrupts. **Next step** is to specify the controller what to do when an interrupt occurs. This is done by writing a subroutine or function for the interrupt. This is the ISR and gets automatically called when an interrupt occurs. It is not required to call the Interrupt Subroutine explicitly in the code.

An important thing is that the definition of a subroutine must have the keyword **interrupt** followed by the interrupt number. A subroutine for a particular interrupt is identified by this number. These subroutine numbers corresponding to different interrupts are tabulated below.

Number	Interrupt	Symbol
0	External0	EX0
1	Timer0	IT0
2	External1	EX1
3	Timer1	IT1
4	Serial	ES
5	Timer2	ET2

For example : Interrupt routine for Timer1

```
void ISR_timer1(void) interrupt 3
{
      <Body of ISR>
}
```

For example : Interrupt routine for External Interrupt0 (EX0)

```
void ISR_ex0(void) interrupt 0
{
      <Body of ISR>
}
```

Note that the interrupt subroutines always have void return type. They never return a value.

Programming Timer Interrupts

The timer interrupts IT0 and IT1 are related to Timers 0 and 1, respectively. (Please refer 8051 Timers for details on Timer registers and modes.) The interrupt programming for timers involves following steps .

1. Configure TMOD register to select timer(s) and its/their mode.
2. Load initial values in THx and TLx for mode 0 and 1; or in THx only for mode 2.
3. Enable Timer Interrupt by configuring bits of IE register.
4. Start timer by setting timer run bit TRx.
5. Write subroutine for Timer Interrupt. The interrupt number is 1 for Timer0 and 3 for Timer1.

   Note that it is not required to clear timer flag TFx.
6. To stop the timer, clear TRx in the end of subroutine. Otherwise it will restart from 0000H in case of modes 0 or 1 and from initial values in case of mode 2.
7. If the Timer has to run again and again, it is required to reload initial values within the routine itself (in case of mode 0 and 1). Otherwise after one cycle timer will start counting from 0000H.

Write a program that continuously get 8-bit data from P0 and sends it to P1 while simultaneously creating a square wave of 200 μs period on pin P2.1. Use timer 0 to create the square wave. Assume that XTAL=11.0592MHz.

Solution:

We will use timer 0 in mode 2 (auto reload). TH0 = 100 μs /1.085 μs = 92

```
                ORG 0000H
                LJMP MAIN

// ISR for timer0 to generate square wave.
                ORG 000BH
                CPL P2.1
                CLR TR0
                RETI

//      Main Program

                ORG 0030H
MAIN:           MOV TMOD, #02H   ; Timer0 in auto reload mode
                MOV  P0, FFH     ; P0 as input port
                MOV TH0, #92d    ; initial value for timer
                MOV IE, #82H     ; enable interrupt
                SETB TR0
BACK:           MOV A, P0
                MOV P1,A
                SJMP BACK

                END
```

Timer interrupt to blink an LED; Time delay in mode1 using interrupt method

```
// Use of Timer mode0 for blinking LED using interrupt method
// XTAL frequency 11.0592MHz
#include<reg51.h>
sbit led = P1^0;          //LED connected to D0 of port 1

void timer(void) interrupt 1          //interrupt no. 1 for Timer
0
{
       led=~led;          //toggle LED on interrupt
       TH0=0xFC;          // initial values loaded to timer
       TL0=0x66;
}
main()
```

```
        TMOD = 0x01;                // model of Timer0
        TH0 = 0xFC;          // initial values loaded to timer
        TL0 = 0x66;
        IE = 0x82;           // enable interrupt
        TR0 = 1;             //start timer
        while(1);            // do nothing
}
```

2. <u>Programming External Interrupts</u>

The external interrupts are the interrupts received from the (external) devices interfaced with the microcontroller. They are received at INTx pins of the controller. These can be level triggered or edge triggered. In level triggered, interrupt is enabled for a low at INTx pin; while in case of edge triggering, interrupt is enabled for a high to low transition at INTx pin. The edge or level trigger is decided by the TCON register.

Setting the IT0 and IT1 bits make the external interrupt 0 and 1 edge triggered respectively. By default these bits are cleared and so external interrupt is level triggered.

Note : For a level trigger interrupt, the INTx pin must remain low until the start of the ISR and should return to high before the end of ISR. If the low at INTx pin goes high before the start of ISR, interrupt will not be generated. Also if the INTx pin remains low even after the end of ISR, the interrupt will be generated once again. This is the reason why level trigger interrupt (low) at INTx pin must be four machine cycles long and not greater than or smaller than this.

Following are the steps for using external interrupt :

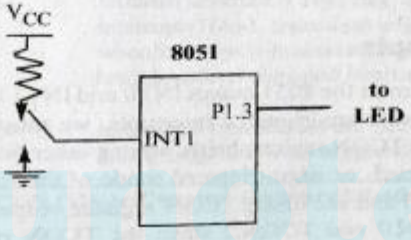
1. Enable external interrupt by configuring IE register.
2. Write routine for external interrupt. The interrupt number is 0 for EX0 and 2 for EX1 respectively.

Assume that the INT1 pin is connected to a switch that is normally high. Whenever it goes low, it should turn on an LED. The LED is connected to P1.3 and is normally off. When it is turned on it should stay on for a fraction of a second. As long as the switch is pressed low, the LED should stay on.

Solution:

```
          ORG   0000H
          LJMP  MAIN              ;bypass interrupt vector table
;--ISR for hardware interrupt INT1 to turn on the LED
          ORG   0013H             ;INT1 ISR
          SETB  P1.3              ;turn on LED
          MOV   R3,#255           ;load counter
BACK:     DJNZ  R3,BACK           ;keep LED on for a while
          CLR   P1.3              ;turn off the LED
          RETI                    ;return from ISR
;--MAIN program for initialization
          ORG   30H
MAIN:     MOV   IE,#10000100B     ;enable external INT1
HERE:     SJMP  HERE              ;stay here until interrupted
          END
```

Pressing the switch will turn the LED on. If it is kept activated, the LED stays on.



Example code#2

```
//Level trigger external interrupt
void main()
{
      IE = 0x81;
      while(1);
}
void ISR_ex0(void) interrupt 0
{
      <body of interrupt>
}
```

Example code#3

```
//Edge trigger external interrupt
void main()
{
      IE = 0x84;
      IT1 = 1;
      while(1);
}
```

```
{
        <body of interrupt>
}
```

3. Programming Serial Interrupt

To use the serial interrupt the ES bit along with the EA bit is set. Whenever one byte of data is sent or received, the serial interrupt is generated and the TI or RI flag goes high. Here, the TI or RI flag needs to be cleared explicitly in the interrupt routine (written for the Serial Interrupt).

The programming of the Serial Interrupt involves the following steps:

1. Enable the Serial Interrupt (configure the IE register).
2. Configure SCON register.
3. Write routine or function for the Serial Interrupt. The interrupt number is 4.
4. Clear the RI or TI flag within the routine.

Example code#1

Example 11-6

Write a program in which the 8051 reads data from P1 and writes it to P2 continuously while giving a copy of it to the serial COM port to be transferred serially. Assume that XTAL = 11.0592 MHz. Set the baud rate at 9600.

Solution:

```
        ORG   0
        LJMP  MAIN
        ORG   23H
        LJMP  SERIAL          ;jump to serial interrupt ISR
        ORG   30H
MAIN:   MOV   P1,#0FFH        ;make P1 an input port
        MOV   TMOD,#20H       ;timer 1, mode 2(auto-reload)
        MOV   TH1,#0FDH       ;9600 baud rate
        MOV   SCON,#50H       ;8-bit, 1 stop, REN enabled
        MOV   IE,#10010000B   ;enable serial interrupt
        SETB  TR1             ;start timer 1
BACK:   MOV   A,P1            ;read data from port 1
        MOV   SBUF,A          ;give a copy to SBUF
        MOV   P2,A            ;send it to P2
        SJMP  BACK            ;stay in loop indefinitely
;
;-----------------Serial Port ISR
        ORG   100H
SERIAL: JB    TI,TRANS        ;jump if TI is high
        MOV   A,SBUF          ;otherwise due to receive
        CLR   RI              ;clear RI since CPU does not
        RETI                  ;return from ISR
TRANS:  CLR   TI              ;clear TI since CPU does not
        RETI                  ;return from ISR
        END
```

In the above program notice the role of TI and RI. The moment a byte is written into SBUF it is framed and transferred serially. As a result, when the last bit (stop bit) is transferred the TI is raised, which causes the serial interrupt to be invoked since the corresponding bit in the IE register is high. In the serial ISR, we check for both TI and RI since both could have invoked the interrupt. In other words, there is only one interrupt for both transmit and receive.

Example code#2

Send ‘A’ from serial port with the use of interrupt

```
// Sending 'A' through serial port with interrupt
// XTAL frequency 11.0592MHz
```

```
{
	TMOD = 0x20;
	TH1 = -1;
	SCON = 0x50;
	TR1 = 1;
	IE = 0x90;
	while(1);
}
void ISR_sc(void) interrupt 4
{
	if(TI==1)
	{
		SBUF = 'A';
		TI = 0;
	}
	else
		RI = 0;
}
```

Example code#3

```
// Receive data from serial port through interrupt
// XTAL frequency 11.0592MHz
void main()
{
	TMOD = 0x20;
	TH1 = -1;
	SCON = 0x50;
	TR1 = 1;
	IE = 0x90;
	while(1);
}
void ISR_sc(void) interrupt 4
{
	unsigned char val;
	if(TI==1)
	{
		TI = 0;
	}
	else
	{
		val = SBUF;
		RI = 0;
	}
}
```

5.4.1. INTERFACING ANLOG TO DIGITAL CONVERTER (ADC) WITH 8051

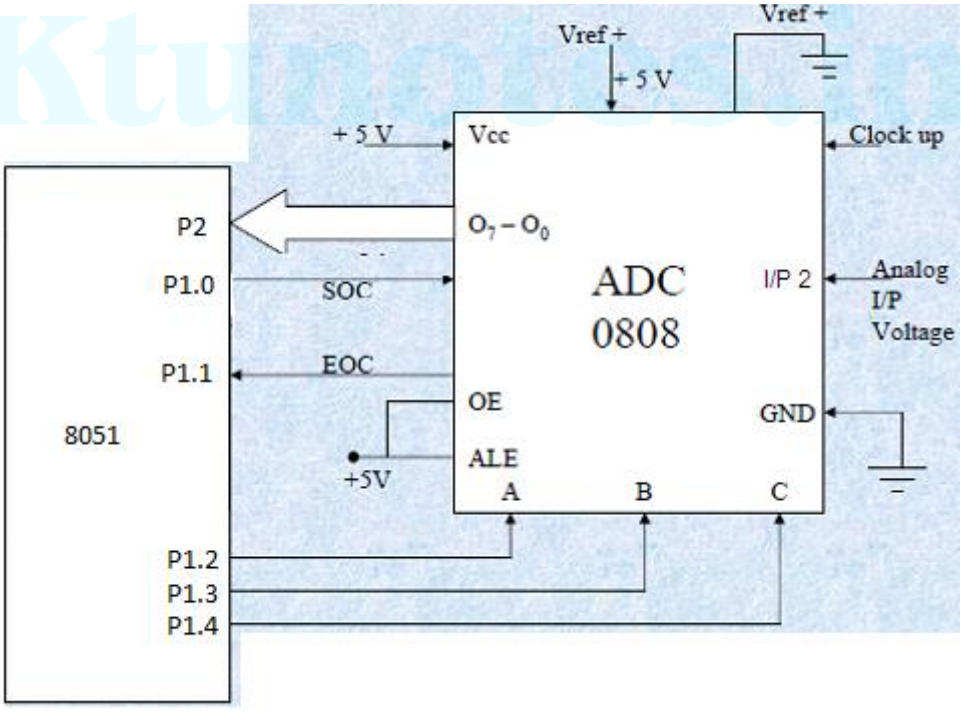
- The analog to digital converter is treated as an input device by the microcontroller.
- ADC 0808 is an ADC chip with 8 input channels which is the commonly used one.

During the analog to digital conversion process,

- Initially, microcontroller sends an initializing signal (start of conversion-SOC) to the ADC to start the analog to digital data conversion process. The start of conversion signal is a pulse of a specific duration.
- The microcontroller has to wait for the digital data till the conversion is over. After the conversion is over, the ADC sends end of conversion EOC signal to inform the microcontroller that the conversion is over and the result is ready at the output buffer of the ADC.

Example: ADC interfacing program to read the digital value output of ADC from Port 2 and send it to Port3.

Solution:



- The analog input I/P2 is used and therefore address pins A,B,C should be 0,1,0 respectively to select I/P2.
- Analog channel select lines are connected at P1.2,P1.3 and P1.4
- Start of conversion signal is connected at P1.0.
- End of conversion signal is connected at P1.1. Hence P1.1 pin is an input pin.
- ADC digital output is connected at Port2. Hence Port 2 is input port.

```
        ORG 0000H
        MOV A,#FFH
        MOV P2,A          ;Initialize P2 as input port
        SETB P1.1         ;Initialize P1.1 as input port

        CLR P1.2      ;Makeselect line a 0
        SETB P1.3         ; Make select line b 1
        CLR P1.4          :Make select line c 0 (abc=010 to select channel)
        CLR P1.0          ; Make SOC low
        SETB P1.0         ; Make SOC high
        CLR P1.0          ; Make SOC low
HERE:   JNB P1.1, HERE
        MOV A,P2          ;Read the digital out from ADC
        MOV P3, A         ; Store at memory.
        END
```

----------Program in C----------

```
#include<REGX51.H>
#define soc P1.0                //soc P1.0
#defineeoc P1.1//eoc P1.1
#define a P1.2//select line a P1.2
#defineb P1.3                   //select line b P1.3
#definec P1.4                   //select line c P1.4

unsignedcharadc_val;

voidmain()
{
P2 = 0xFF;                      //Initialize Port2 as input port
eoc = 1;                        //Initialize Port1.1 as input pin

      while(1)                  //Forever loop
      {
      a =0;//Make a low
      b =1;//Make b high
      c =0;//Make c low (abc=010 to select channel)
      soc =0;//Make soc low
      soc =1;//Make soc high
      soc =0;//Make soc low
      while(eoc==1);//Wait for eoc to go high
      adc_val = P2;   //Read digital value from P2
      P3 =adc_val;//Send the digital value to P3
      }
}
```

- In the DAC0808, the digital inputs are converted to current (I_{out}), and by connecting a resistor to the I_{out} pin, we convert the result to voltage.
- The total current provided by the I_{out} pin is a function of the binary numbers at the DO – D7 inputs of the DAC0808 and the reference current (I_{ref}), and is as follows:

$$I_{out} = I_{ref}\left(\frac{D7}{2} + \frac{D6}{4} + \frac{D5}{8} + \frac{D4}{16} + \frac{D3}{32} + \frac{D2}{64} + \frac{D1}{128} + \frac{D0}{256}\right)$$

where DO is the LSB, D7 is the MSB for the inputs, and I_{ref} is the input current that must be applied to pin 14

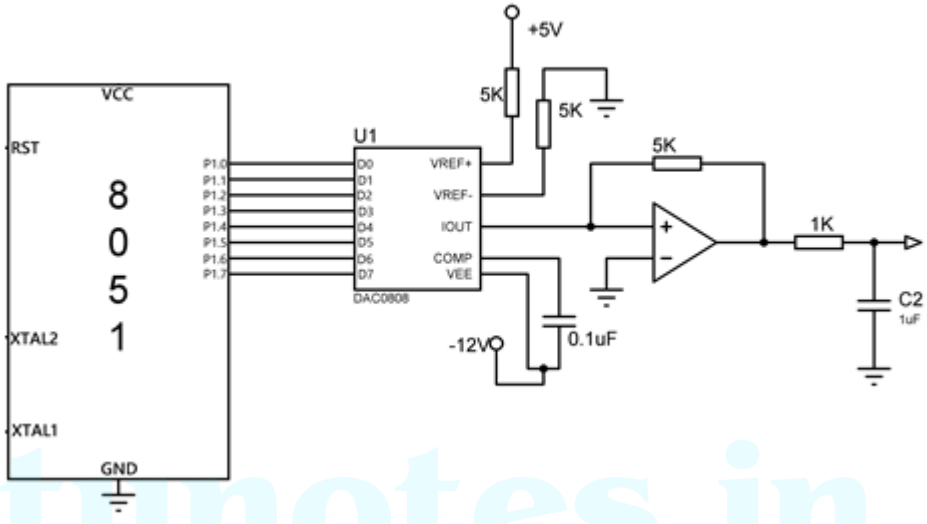


Figure: Interfacing DAC with 8051

Programming Example:

Write an ALP to generate Triangular wave form on port P1 of 8051 microcontroller using DAC.

```
        ORG 0000h
        MOV P1, #00H
REPEAT: ACALL TRIWAVE
        SJMP REPEAT

TRIWAVE: MOV A.#00
INCR:   MOV P1,A
        INC A
        CJNE A,#0FFH,INCR
DECR:   MOV P1,A
        DEC A
        CJNE A,#00H,DECR
        RET
        END
```

Interfacing 8051 with ADC

Introduction

- The physical parameters that any microcontroller processes come from sensors.
- Sensors are transducers that convert a physical parameter like temperature into electrical signals that the microcontroller can understand.
- But here is the issue—analog sensors output data in an analog format which a microcontroller cannot understand.
- Therefore, to convert this analog data to a digital format, Analog to Digital converters or ADCs are used.

What is an ADC?

- An analog to digital converter or ADC, as the name suggests, converts an analog signal to a digital signal.
- An analog signal has a continuously changing amplitude with respect to time.
- A digital signal, on the contrary, is a stream of 0s and 1s.
- An ADC maps analog signals to their binary equivalents.
- To do this, ADCs use various methods like Flash conversion, slope integration, or successive approximation.

- To understand the ADC in a better way, let us look at an example.
- Let us say we have an input signal which varies from 0 to 8 volt, and we use a 3-bit ADC to convert this signal to binary data.
- A 3-bit ADC can represent 2^3 or 8 different voltage levels using 3 bits of data.

Input voltage	Binary equivalent
0-1 volt	000
1-2 volt	001
2-3volt	010
3-4 volt	011
4-5 volt	100
5-6 volt	101
6-7 volt	110
7-8 volt	111

- From the table above, we can see how the ADC maps analog data to digital values.
- In the case mentioned above, we can see that the tiniest change we can detect is that of 1 volt.
- If the change is smaller than 1 volt, the ADC can't detect it.
- This minimum change that an ADC can detect is known as the step size of the ADC.
- To calculate it, we can use the formula:

$$\text{Step size} = (V_{max} - V_{min})/2^n$$

(where n is the number of bits(resolution) of an ADC)

- The step size of an ADC is inversely proportional to the number of bits of an ADC.
- So using an ADC with higher bits can detect smaller changes, but this increases the cost of production.
- Due to this reason, most on-chip ADCs' have an 8-bit/10-bit resolution.
- Given below is the resolution vs. step size for various configurations with a range of 0-5v input signal.

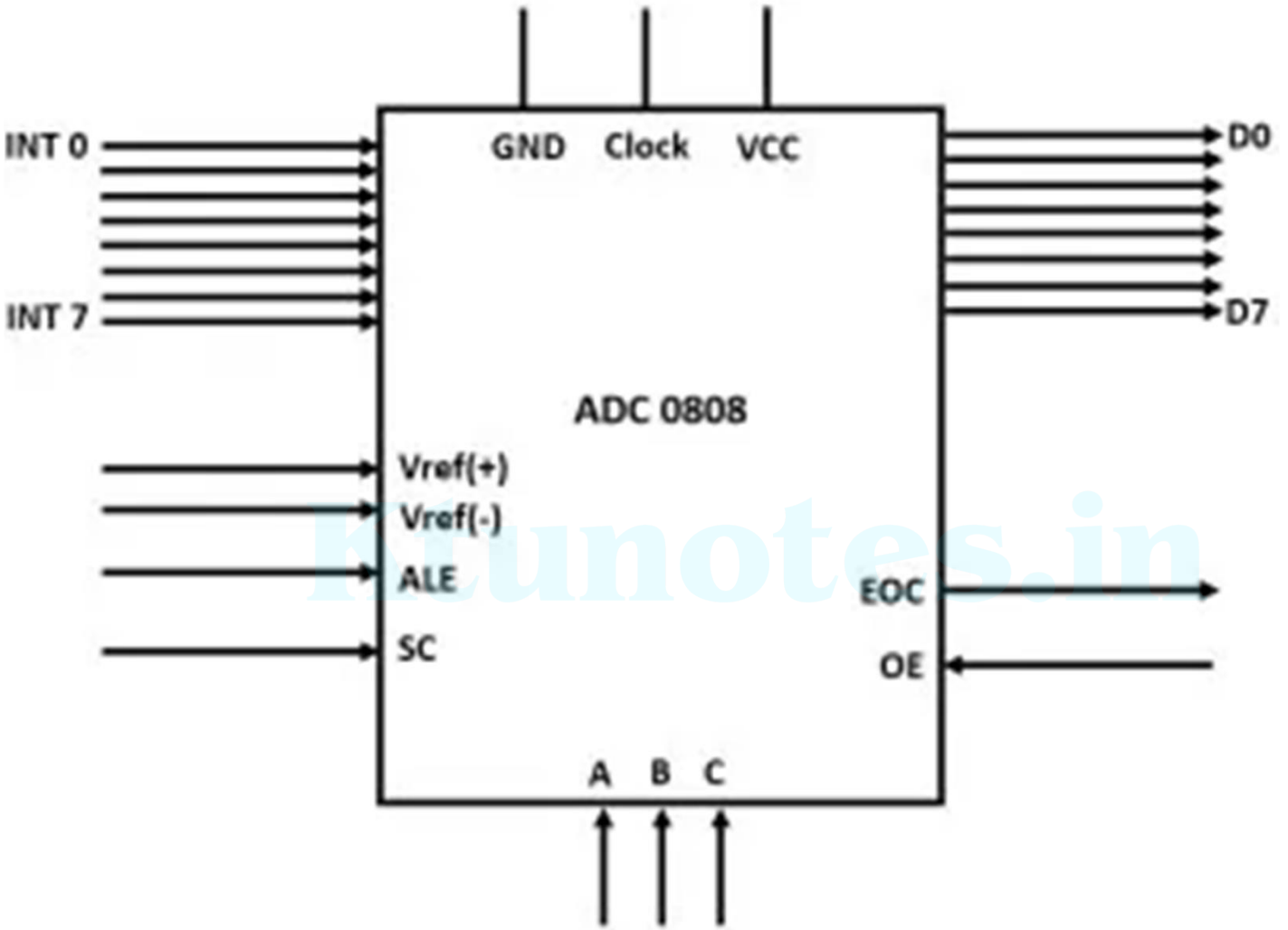
Number of bits	Number of steps	step size(mV)
8	256	5/256=19.53
10	1024	5/1024=4.88
12	4096	5/4096=1.2
16	65536	0.076 (precise conversion)

ADC0808

- One of the most commonly used ADC is ADC0808
- ADC 0808 is a Successive approximation type with 8 channels i.e. it can directly access 8 single-ended analog signals.
- **ADC0808** is an 8 bit **analog to digital converter** with eight input analog channels, i.e., it can take eight different analog inputs.
- The voltage reference can be set using the Vref+ and Vref- pins.

ADC0808

- The step size is decided based on the set reference value. Step size is the change in analog input to cause a unit change in the output of ADC. The default step size is 19.53mV corresponding to 5V reference voltage.
- ADC0808 needs an external clock to operate.
- The ADC needs some specific control signals for its operations like start conversion and brings data to output pins.
- When the conversion is complete the EOC pins go low to indicate the end of conversion and data ready to be picked up.


GND
Clock
VCC
INT 0
INT 7
D0
D7
ADC 0808
Vref(+)
Vref(-)
ALE
SC
EOC
OE
A
B
C

Input pins (INT0-INT7)

- The ADC 0808 has eight input analog pins. These pins are multiplexed together, and only one of them can be selected using three select lines.

Select lines and ALE

- It has three select lines, namely A, B, and C, that are used to select the desired input lines. The ALE pin also needs to be activated by a low to high pulse to select a particular input. The input lines are selected as follows:

A	B	C	Selected analog channel	ALE pin
0	0	0	INT0	Low to High pulse
0	0	1	INT1	Low to High pulse
0	1	0	INT2	Low to High pulse
0	1	1	INT3	Low to High pulse
1	0	0	INT4	Low to High pulse
1	0	1	INT5	Low to High pulse
1	1	0	INT6	Low to High pulse
1	1	1	INT7	Low to High pulse

Output pins (D0-D7)

- The ADC has eight output pins that give the binary equivalent of a given analog value.

VCC and Ground

- These two pins are used to provide the required voltage to power the microcontroller. In most cases, the ADC uses 5V DC to power up.

Clock

- As mentioned earlier, the 0808 does not have an internal clock and needs an external clock signal to operate. It uses a clock frequency of 20Mhz, and using this clock frequency it can perform one conversion in 100 microseconds.

VREF (+) and VREF (-)

- These two pins are used to provide the upper and the lower limit of voltages which determine the step size for the conversion. Here Vref(+) has a higher voltage, and Vref(-) has the lower voltage.
- If Vref(+) has an input voltage 5v and Vref(-) has a voltage of 0v then the step size will be $5v-0v/2^8 = 15.53$ mv.

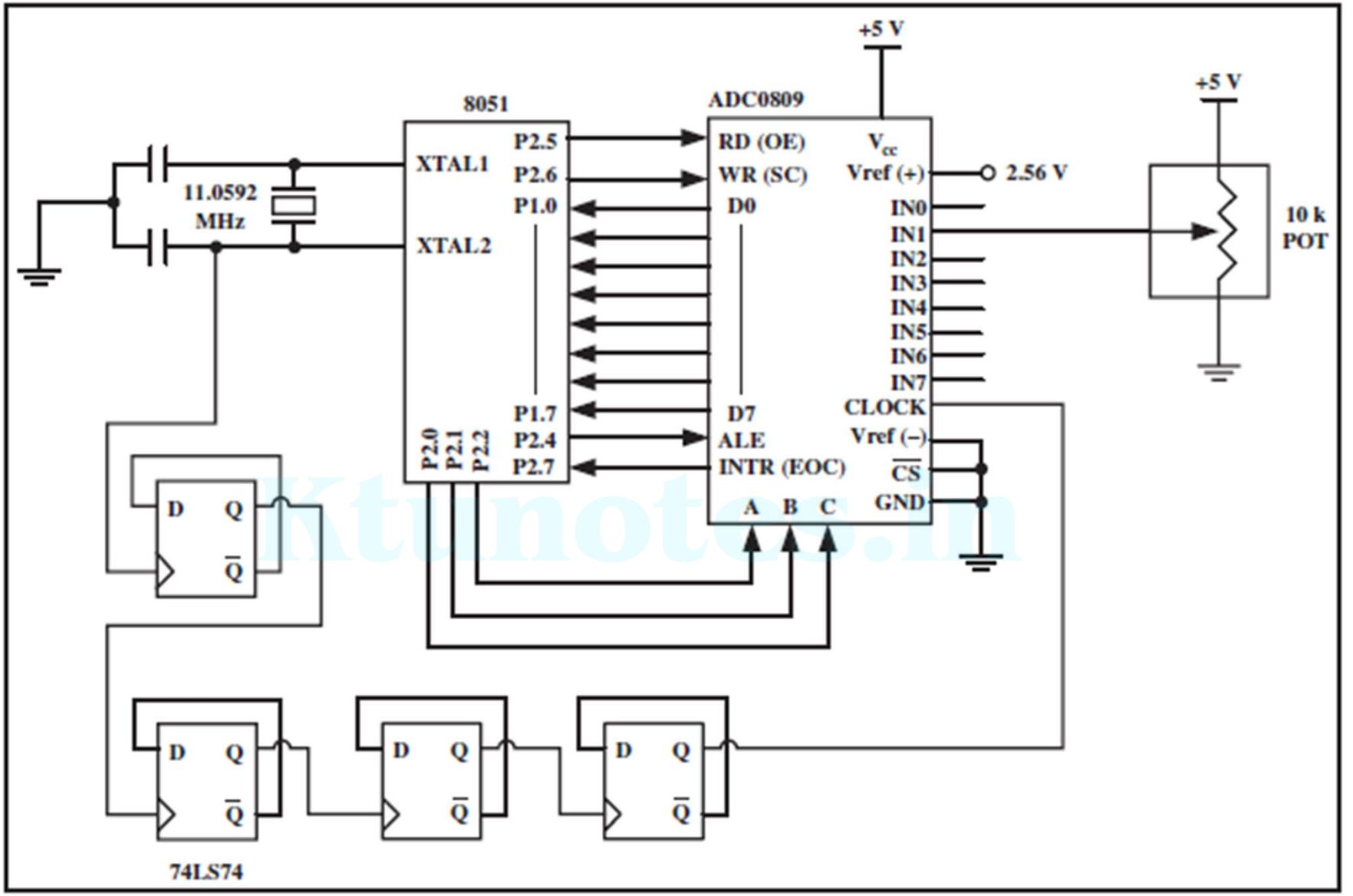
End of conversion

- Once the conversion is complete, the ADC sends low to high signal to tell a microcontroller that the conversion is complete and that it can extract the data from the 8 data pins.

Output enable

- This pin is used to extract the data from the ADC. A microcontroller sends a low to high pulse to the ADC to extract the data from its data buffers

- **Steps to program the ADC0808/0809**
- The following are steps to get data from an ADC0808/0809.
- 1. Select an analog channel by providing bits to A, B, and C addresses
- according to Table 3.
- 2. Activate the ALE (address latch enable) pin. It needs an L-to-H pulse to
- latch in the address. See Figure 6.
- 3. Activate SC (start conversion) by an L-to-H pulse to initiate conversion.
- 4. Monitor EOC (end of conversion) to see whether conversion is finished.
- H-to-L output indicates that the data is converted and is ready to be picked
- up. If we do not use EOC, we can read the converted digital data after a
- brief time delay. The delay size depends on the speed of the external clock
- we connect to the CLK pin. Notice that the EOC is the same as the INTR
- pin in other ADC chips.
- 5. Activate OE (output enable) to read data out of the ADC chip. An L-to-H
- pulse to the OE pin will bring digital data out of the chip. Also notice that
- the OE is the same as the RD pin in other ADC chips.
- Notice in the ADC0808/0809 that there is no self-clocking and the clock
- must be provided from an external source to the CLK pin. Although the speed
- of conversion depends on the frequency of the clock connected to the CLK
- pin, it cannot be faster than 100 microseconds.


+5 V
8051
ADC0809
+5 V
XTAL1
XTAL2
11.0592
MHz
P2.5
P2.6
P1.0
P1.7
P2.0
P2.1
P2.2
P2.4
P2.7
RD (OE)
WR (SC)
D0
D7
ALE
INTR (EOC)
A
B
C
V_{cc}
Vref (+)
2.56 V
IN0
IN1
IN2
IN3
IN4
IN5
IN6
IN7
CLOCK
Vref (–)
$\overline{CS}$
GND
10 k
POT
D
Q
$\overline{Q}$
74LS74

Programming ADC0808/0809 in Assembly

```
        ALE    BIT P2.4
        OE     BIT P2.5
        SC     BIT P2.6
        EOC    BIT P2.7
        ADDR_A BIT P2.0
        ADDR_B BIT P2.1
        ADDR_C BIT P2.2
        MYDATA EQU P1
        ORG    0H
        MOV    MYDATA,#0FFH     ;make P1 an input
        SETB   EOC              ;make EOC an input
        CLR    ALE              ;clear ALE
        CLR    SC               ;clear WR
        CLR    OE               ;clear RD
BACK:
        CLR    ADDR_C           ;C=0
        CLR    ADDR_B           ;B=0
        SETB   ADDR_A           ;A=1 (Select Channel 1)
        ACALL  DELAY            ;make sure the addr is stable
        SETB   ALE              ;latch address
        ACALL  DELAY            ;delay for fast DS89C4x0 Chip
        SETB   SC               ;start conversion
        ACALL  DELAY
        CLR    ALE
        CLR    SC
```

```
HERE:
        JB      EOC, HERE           ;wait until done
HERE1:
        JNB     EOC, HERE1          ;wait until done
        SETB    OE                  ;enable RD
        ACALL   DELAY               ;wait
        MOV     A,MYDATA            ;read data
        CLR     OE                  ;clear RD for next time
        ACALL   CONVERSION          ;hex to ASCII (Chap 6)
        ACALL   DATA_DISPLAY        ;display the data (Chap 12)
        SJMP    BACK
```

Programming ADC0808/0809 in C

```
#include <reg51.h>
sbit ALE = P2^4;
sbit OE = P2^5;
sbit SC = P2^6;
sbit EOC = P2^7;
sbit ADDR_A = P2^0;
sbit ADDR_B = P2^1;
sbit ADDR_C = P2^2;
sfr  MYDATA = P1;
```

```
void main()
  {
    unsigned char value;
    MYDATA = 0xFF;                    //make P1 an input
    EOC = 1;                          //make EOC an input
    ALE = 0;                          //clear ALE
    OE = 0;                           //clear OE
    SC = 0;                           //clear SC
    while(1)
      {
        ADDR_C = 0;                   //C=0
        ADDR_B = 0;                   //B=0
        ADDR_A = 1;                   //A=1 (Select Channel 1)
        MSDelay(1);                   //delay for fast DS89C4x0
        ALE = 1;
        MSDelay(1);
        SC = 1;
        MSDelay(1);
        ALE = 0;
        SC = 0;                       //start conversion
        while(EOC==1);                //wait for data conversion
        while(EOC==0);
        OE = 1;                       //enable RD
        MSDelay(1);
        value = MYDATA;               //get the data
        OE = 0;                       //disable RD for next round
        ConvertAndDisplay(value);     //Chap 7 & 12
      }
```

DAC INTERFACING

Digital-to-analog (DAC) converter

The digital-to-analog converter (DAC) is a device widely used to convert digital pulses to analog signals. There are two methods of creating a DAC: binary weighted and R/2R ladder.

The vast majority of integrated circuit DACs, including the MC1408 (DAC0808) use

the R/2R method since it can achieve a much higher degree of precision.

The first criterion for judging a DAC is its resolution, which is a function of the number of binary inputs.

The common ones are 8, 10, and 12 bits.

DAC INTERFACING

Digital-to-analog (DAC) converter

The number of data bit inputs decides the resolution of the DAC since the number of analog output levels is equal to 2*n*, where *n* is the number of data bit inputs.

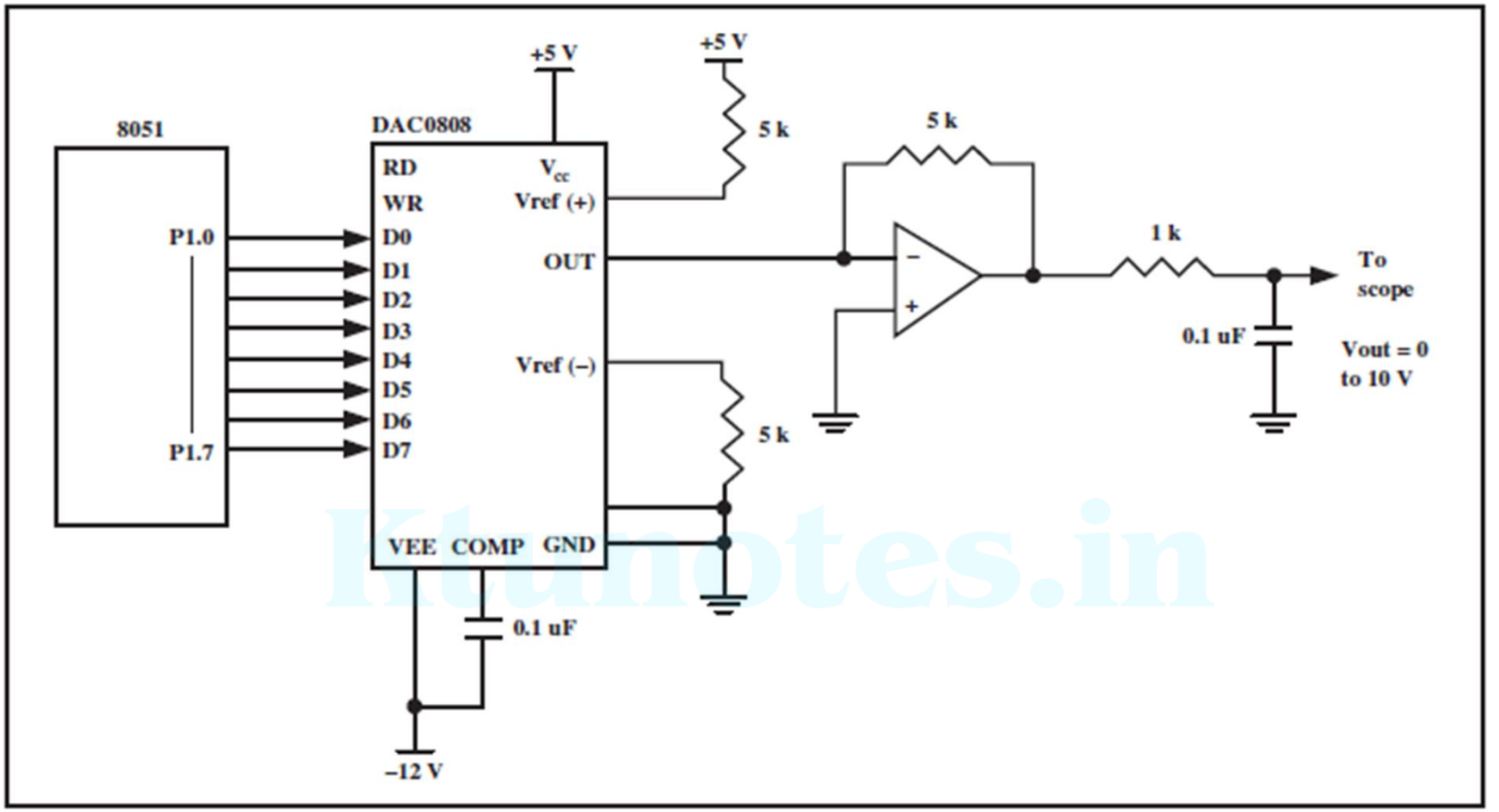
Therefore, an 8-input DAC such as the DAC0808 provides 256 discrete voltage (or current) levels of output. Similarly, the 12-bit DAC provides 4096 discrete voltage levels. There are also 16-bit DACs, but they are more expensive.

MC1408 DAC (or DAC0808)

- In the MC1408 (DAC0808), the digital inputs are converted to current (Iout), and by connecting a resistor to the Iout pin, we convert the result to voltage.
- The total current provided by the Iout pin is a function of the binary numbers at the D0–D7 inputs of the DAC0808 and the reference current (Iref), and is as follows:

$$I_{out} = I_{ref}\left(\frac{D7}{2} + \frac{D6}{4} + \frac{D5}{8} + \frac{D4}{16} + \frac{D3}{32} + \frac{D2}{64} + \frac{D1}{128} + \frac{D0}{256}\right)$$

- Here, D0 is the LSB, D7 is the MSB for the inputs, and Iref is the input current that must be applied to pin 14. The Iref current is generally set to 2 mA.
- Fig. shows the generation of current reference (setting Iref = 2 mA) by using the standard 5 V power supply and 1 K and 1.5 K-ohm standard resistors.
- Some DACs also use the zener diode (LM336), which overcomes any fluctuation associated with the power supply voltage.


8051
P1.0
P1.7
DAC0808
RD
WR
D0
D1
D2
D3
D4
D5
D6
D7
VEE
COMP
V_{cc}
Vref (+)
OUT
Vref (–)
GND
+5 V
+5 V
5 k
5 k
5 k
1 k
0.1 uF
0.1 uF
To
scope
Vout = 0
to 10 V
–12 V

Assuming that R = 5K and I_{ref} = 2 mA, calculate V_{out} for the following binary inputs:
(a) 10011001 binary (99H) (b) 11001000 (C8H).

Solution:

(a) I_{out} = 2 mA (153/256) = 1.195 mA and V_{out} = 1.195 mA × 5K = 5.975 V
(b) I_{out} = 2 mA (200/256) = 1.562 mA and V_{out} = 1.562 mA × 5K = 7.8125 V

- Write as example program to create triangular/sawtooth wave done in lab

8051 Time Delays

- 8051 Timer Clock Frequency : The frequency of the 8051 timer is always $(1/12)^{th}$ of the frequency of the crystal connected to 8051.

- Example : If Crystal frequency (XTAL Oscillator) is 24 MHz, then the frequency of the 8051 Timer Clock is $(1/12)^{th}$ of crystal frequency, i.e., 2 MHz.

- Then, the time period of the Timer Clock is 1/(timer frequency) = 1/(2 MHz) = 0.5 μs = 500 ns

- Note : XTAL frequency of 11.0592 MHz is normally used for 8051 microcontroller. The reason behind such an odd number has to do with the baud rate for serial communication of the 8051.

- XTAL = 11.0592 MHz allows the 8051 system to communicate with the IBM PC with no errors.

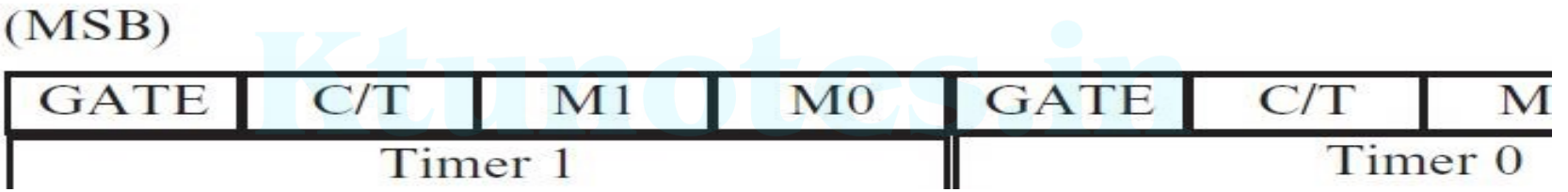
8051 Time Delays

- 8051 has two Timers – Timer 0 and Timer 1, which are used to generate time delays.
- Both T0 and T1 are 16-bit size. Each timer is accessed as two separate 8-bit registers (One low byte and one high byte).
- Timer 0 : **TL0** Register - Timer 0 low byte and **TH0** Register - Timer 0 high byte.
- Timer 1 : **TL1** Register - Timer 1 low byte and **TH1** Register - Timer 1 high byte.
- TH1, TL1, TH0 and TL0 registers can be used similar to other registers such as A, B, R1, R0, etc.

- Ex: MOV TL0, #4FH
- Ex: MOV TH1, A

8051 Time Delays

- **TMOD** : It is an 8-bit register that is used to set various timer operation modes.
- The lower 4-bits are set aside for Timer 0 and higher 4-bits are set aside for Timer 1 as shown below.



- M1 and M0 selects the timer modes (4 modes are present).
- C/T decides whether the timer is uses as delay generator (C/T = 0) or event counter (c/T = 1).
- GATE bit decides whether the Timers are started and stopped using software (GATE = 0) or hardware (GATE = 1).
- Software START / STOP is done using instructions SETB and CLR.

8051 Time Delays

Timer	Software Starting	Software Stopping
Timer 0	SETB TR0	CLR TR0
Timer 1	SETB TR1	CLR TR1

Mode	M1	M0	Details of Mode
0	0	0	13-bit Timer Mode
1	**0**	**1**	**16-bit Timer Mode (Normally Used)**
2	1	0	8-bit Timer Mode
3	1	1	Split Timer Mode

- Example : MOV TMOD, #01 = Timer 0 is SET as Mode 1
- Example : MOV TMOD, #20 = Timer 1 is SET as Mode 2

8051 Time Delays

- **MODE 1 Timer Programming**
- Both Timer0 and Timer1 can be operated in MODE 1 as 16-bit timers.
- In **MODE 1**, Timer can allow values from 0000 to FFFF to be loaded to TH and TL registers for implementing various time delays.
- **Steps to Implement MODE 1 Timer**

- Select the Timer(0 or 1) and load TMOD register with required value.
- Load the 16-bit count value into the TH and TL registers.
- Start the selected Timer using the instruction SETB TR0 / SETB TR1
- Once the Timer is started, it will start counting up from the set value in TH and TL registers to FFFF. Once the count reaches FFFF, it will roll over to 0000 and will set **timer flag high**.
- By checking the timer flag, stop the counter using CLR TR0 / CLR TR1. The Timer flag can be checked by JNB TF0 / JNB TF1 instrution.

8051 Time Delays

- **Time Delay Count Calculation**
- If we know the required amount of time delay to be generated, we can find the delay count to be loaded into the Timer registers TH and TL in Mode 1.
- Let the XTAL Frequency = 11.0592 MHz,
- Then the Timer Clock Frequency = 0.9216 MHz
- The Timer Clock Time period = 1.085 µs (Time taken to execute Timer Subroutine one time)
- Let the time delay required be 5 ms. Then the required number of times the Timer subroutine need to be executed = 5ms / 1.085 µs = 4608_{10}.
- Subtract 4608 from 65536 and convert the result into hexadecimal to get the value to be loaded into the TH and TL Registers.
- 65536 – 4608 = 60928 = $EE00_H$. Load EE to TH and 00 to TL registers

8051 Time Delays – Program Example 1

Example 10

Assume that XTAL = 11.0592 MHz. What value do we need to load into the timer's registers if we want to have a time delay of 5 ms? Show the program for Timer 0 to create a pulse width of 5 ms on P2.3.

Solution:

Since XTAL = 11.0592 MHz, the counter counts up every 1.085 μs. This means that out of many 1.085 μs intervals we must make a 5 ms pulse. To get that, we divide one by the other. We need 5 ms / 1.085 μs = 4608 clocks. To achieve that we need to load into TL and TH the value 65536 – 4608 = 60928 = EE00H. Therefore, we have TH = EE and TL = 00.

```
            CLR   P2.3              ;clear P2.3
            MOV   TMOD,#01          ;Timer 0, mode 1 (16-bit mode)
HERE:       MOV   TL0,#0            ;TL0 = 0, Low byte
            MOV   TH0,#0EEH         ;TH0 = EE( hex), High byte
            SETB  P2.3              ;SET P2.3 high
            SETB  TR0               ;start Timer 0
AGAIN:      JNB   TF0,AGAIN         ;monitor Timer 0 flag
                                    ;until it rolls over
            CLR   P2.3              ;clear P2.3
            CLR   TR0               ;stop Timer 0
            CLR   TF0               ;clear Timer 0 flag
```

8051 Time Delays – Program Example 2

Example 11

Assuming that XTAL = 11.0592 MHz, write a program to generate a square wave of 2 kHz frequency on pin P1.5.

Solution:

This is similar to Example 10, except that we must toggle the bit to generate the square wave. Look at the following steps.
(a) T = 1 / f = 1 / 2 kHz = 500 μs the period of the square wave.
(b) 1/2 of it for the high and low portions of the pulse is 250 μs.
(c) 250 μs / 1.085 μs = 230 and 65536 – 230 = 65306, which in hex is FF1AH.
(d) TL = 1AH and TH = FFH, all in hex. The program is as follows.

```
            MOV   TMOD,#10H        ;Timer 1, mode 1(16-bit)
AGAIN:      MOV   TL1,#1AH         ;TL1=1AH, Low byte
            MOV   TH1,#0FFH        ;TH1=FFH, High byte
            SETB  TR1              ;start Timer 1
BACK:       JNB   TF1,BACK         ;stay until timer rolls over
            CLR   TR1              ;stop Timer 1
            CPL   P1.5             ;complement P1.5 to get hi, lo
            CLR   TF1              ;clear Timer 1 flag
            SJMP  AGAIN            ;reload timer since mode 1
                                   ;is not auto-reload
```

8051 Time Delays – Program Example 3

Example 12

Assuming XTAL = 11.0592 MHz, write a program to generate a square wave of 50 Hz frequency on pin P2.3.

Solution:

Look at the following steps.
(a) T = 1 / 50 Hz = 20 ms, the period of the square wave.
(b) 1/2 of it for the high and low portions of the pulse = 10 ms
(c) 10 ms / 1.085 μs = 9216 and 65536 – 9216 = 56320 in decimal, and in hex it is DC00H.
(d) TL = 00 and TH = DC (hex)

The program follows.

```
            MOV   TMOD,#10H        ;Timer 1, mode 1 (16-bit)
AGAIN:      MOV   TL1,#00          ;TL1 = 00, Low byte
            MOV   TH1,#0DCH        ;TH1 = DCH, High byte
            SETB  TR1              ;start Timer 1
BACK:       JNB   TF1,BACK         ;stay until timer rolls over
            CLR   TR1              ;stop Timer 1
            CPL   P2.3             ;comp. P2.3 to get hi, lo
            CLR   TF1              ;clear Timer 1 flag
            SJMP  AGAIN            ;reload timer since mode 1
                                   ;is not auto-reload
```